

**Informe técnico de la Versión 3: arquitectura distribuida para el cálculo de velocidades del
SITM-MIO**

María José Betancourt Solano - A00403987

Joshua Sayur Gallego - A00404707

Martin Borrero Herrera - A00403871

Javier Andrés Paz Luna - A00405619

Camilo Andrés Martínez Moreno - A00405205

Universidad Icesi

Ingeniería de Software IV

Proyecto final

Informe técnico de la Versión 3: arquitectura distribuida para el cálculo de velocidades del SITM-MIO

Objetivo

El propósito de la Versión 3 fue llevar el cálculo de velocidades promedio del SITM-MIO a un escenario distribuido. En las versiones anteriores el procesamiento se ejecutaba en una sola máquina; en esta versión, el trabajo se divide entre varios equipos para responder mejor a los atributos de calidad de rendimiento, escalabilidad, correctitud y tolerancia operacional ante datasets más grandes.

La funcionalidad central se conserva: calcular la velocidad promedio de los buses por ruta y por mes a partir de registros GPS. La diferencia principal está en la forma de ejecutar el proceso: el sistema se compone de cliente, broker, master, workers, particionador y visualización, comunicados mediante ZeroC Ice. Ice se utiliza como middleware de comunicación remota orientado a objetos, lo cual permite invocar servicios distribuidos a través de interfaces definidas previamente (ZeroC, 2018).

Método de cálculo

Datos de entrada

La solución usa dos archivos de entrada. El primero es datagrams4Pilot.csv, que contiene los registros GPS de los buses. El segundo es lines-241-ActiveGT.csv, que lista las rutas activas que deben considerarse. Antes de procesar los datagramas, cada worker carga las rutas activas para descartar registros que no pertenecen al subconjunto solicitado.

Tabla 1

Columns utilizadas del archivo de datagramas

Índice	Campo	Descripción
[4]	latitude	Latitud GPS guardada como entero; se divide entre 1e7 para obtener grados decimales.
[5]	longitude	Longitud GPS guardada como entero; se divide entre 1e7 para obtener grados decimales.
[7]	lineId	Identificador de la ruta en la que se encuentra el bus.
[10]	datagramDate	Fecha y hora del datagrama con formato yyyy-MM-dd HH:mm:ss.
[11]	busId	Identificador del bus.

Nota. Estas columnas permiten identificar la ruta, el bus, la posición geográfica y el momento en que se produjo cada registro.

Descripción del algoritmo

El primer paso consiste en dividir el dataset en particiones. Para no perder correctitud, la partición se realiza con la clave busId|lineId. Esta decisión es crítica: si los registros de un mismo bus y una misma ruta quedaran separados en workers distintos, se rompería la comparación entre posiciones consecutivas y el cálculo de velocidad podría quedar incompleto o sesgado.

Cada worker recibe una partición y la recorre de forma secuencial. En lugar de guardar todas las posiciones en memoria, conserva únicamente la última posición conocida para cada combinación busId|lineId. Cuando llega una nueva posición del mismo grupo, calcula el tiempo transcurrido, la distancia recorrida y la velocidad. Este enfoque reduce el consumo de memoria y es más adecuado para un dataset piloto de mayor tamaño.

El cálculo se hace primero por día y luego por mes. Las velocidades válidas se acumulan por ruta y día; después, esos acumulados diarios se consolidan para obtener el promedio mensual por ruta. Se descartan los pares de puntos con tiempo menor o igual a cero, distancia menor o igual a cero o velocidad superior a 80 km/h. El descarte de velocidades superiores al umbral evita que errores de GPS, saltos de posición o registros inconsistentes distorsionen el promedio final.

Fórmula Haversine

La distancia entre dos coordenadas GPS se calcula mediante la fórmula Haversine, usada comúnmente para estimar distancias de círculo máximo entre dos puntos sobre una esfera (Sinnott, 1984):

$$a = \sin^2(\Delta\text{lat}/2) + \cos(\text{lat1}) \times \cos(\text{lat2}) \times \sin^2(\Delta\text{lon}/2)$$

$$\text{distancia} = 6371 \times 2 \times \text{atan2}(\sqrt{a}, \sqrt{1-a}) \text{ [km]}$$

En esta fórmula, las latitudes y longitudes se expresan en radianes. El valor 6371 corresponde al radio aproximado de la Tierra en kilómetros. La fórmula es suficiente para el objetivo del proyecto, aunque no modela con precisión el elipsoide terrestre; por tanto, su resultado debe entenderse como una aproximación práctica, no como una medición geodésica exacta.

Optimizaciones aplicadas

La Versión 3 no solo distribuye el procesamiento, sino que también ajusta la forma en que cada worker maneja la información. Esto fue necesario porque el dataset piloto es más grande que el utilizado para las pruebas iniciales y cargar todas las posiciones en memoria podía producir errores de memoria.

Tabla 2

Optimizaciones implementadas en la Versión 3

Optimización	Descripción
Particionamiento por bus y ruta	Mantiene juntas las posiciones que deben compararse para calcular velocidades.
Procesamiento distribuido	Reparte particiones entre varios workers para reducir el trabajo individual por equipo.
Procesamiento por streaming	Evita guardar listas completas de datagramas; cada worker conserva solo la última posición por grupo y acumulados parciales.
Acumulación diaria y consolidación mensual	Calcula primero por ruta y día, y luego consolida por ruta y mes.
Visualización filtrada	Permite observar eventos y recorridos sin saturar el mapa.

Nota. Las optimizaciones se enfocan en rendimiento, uso de memoria y escalabilidad sin cambiar el resultado funcional esperado.

Resultados

La ejecución distribuida se validó con diez workers. El cliente envía la tarea al broker, el broker la enruta al master y el master reparte diez particiones entre los workers registrados. Cada worker procesa su partición y devuelve estadísticas parciales, que luego son consolidadas por el master.

Tabla 3

Evidencias de la ejecución distribuida

Elemento	Evidencia observada
Workers	Registro de workers en el master y procesamiento de 10/10 particiones.
Cliente	Finalización de la tarea con Success: true.
Salida	Archivo output/resultados-v3.csv reportado por el flujo distribuido.
Registros	1356 registros generados en la salida final.
Tiempo reportado	65 429 ms en la ejecución mostrada en consola.

Nota. Los valores corresponden a la ejecución evidenciada en la captura de consola incluida en la Figura 1.

Comparación con las versiones anteriores

La comparación debe leerse con cautela: V1 y V2 fueron medidas con datagrams-MiniPilot.csv y generaron 97 registros, mientras que V3 se ejecutó con datagrams4Pilot.csv particionado y generó 1356 registros. Por tanto, no es técnicamente correcto afirmar que V3 es “más rápida” o “más lenta” solo por el tiempo total; el valor de V3 está principalmente en escalabilidad, distribución y control de memoria.

Tabla 4*Comparación técnica entre V1, V2 y V3*

Versión	Arquitectura	Dataset / salida	Tiempo reportado	Lectura técnica
V1	Monolítica MVC secuencial	MiniPilot / 97 registros	13 045 ms	Base funcional simple; fácil de validar, pero limitada en rendimiento.
V2	Monolítica MVC con concurrencia local	MiniPilot / 97 registros	11 128 ms	Mejora el cálculo local usando hilos, sin distribuir el sistema.
V3	Distribuida con Broker y Master-Worker	Pilot particionado / 1356 registros	65 429 ms	Aumenta escalabilidad y reduce presión de memoria por worker, aunque agrega overhead de red y coordinación.

Nota. Los tiempos no son una comparación experimental estricta porque las versiones no usaron exactamente el mismo dataset ni las mismas condiciones de ejecución.

Frente a V1, la Versión 3 cambia el problema de un procesamiento secuencial centralizado a un flujo distribuido. Frente a V2, ya no se trata solo de paralelismo dentro de una máquina, sino de coordinación entre nodos. Esta diferencia introduce costos adicionales: registro de workers, comunicación remota, serialización de resultados, espera por particiones y consolidación en el master. Sin embargo, esos costos son aceptables cuando el tamaño del dataset crece y una sola máquina deja de ser suficiente.

Evidencia de ejecución en consola

Figura 1

Ejecución distribuida de la Versión 3 con worker y cliente

```

swarch@104m02:~/Documents/JCMMJ/data$ cd /home/swarch/Documents/JCMMJ
ls data/partitions-10
partition-0.csv
swarch@104m02:~/Documents/JCMMJ$ ls data/lines-241-ActiveGT.csv
data/lines-241-ActiveGT.csv
swarch@104m02:~/Documents/JCMMJ$ ls data/partitions-10/partition-0.csv
data/partitions-10/partition-0.csv
swarch@104m02:~/Documents/JCMMJ$ bash v3-distributed/start-node.sh worker 1
Jun 07, 2026 8:39:28 PM co.icesi.project.v3.worker.WorkerServer main
INFO: worker-1 ready at tcp -h 192.168.131.102 -p 11011
Jun 07, 2026 8:39:28 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 1)
Jun 07, 2026 8:39:46 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 10)
Jun 07, 2026 8:40:06 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 20)
Jun 07, 2026 8:40:26 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 30)
Jun 07, 2026 8:40:47 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 40)
Jun 07, 2026 8:41:07 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 50)
Jun 07, 2026 8:41:27 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 60)
Jun 07, 2026 8:41:47 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 70)
Jun 07, 2026 8:42:07 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 waiting for Master at Master:tcp -h 192.168.131.101 -p 11001 (attempt 80)
Jun 07, 2026 8:42:13 PM co.icesi.project.v3.worker.WorkerServer lambda$main$0
INFO: worker-1 registered with Master

swarch@104m31:~/Documents/JCMMJ/v3-distributed/client$ ls
ls
swarch@104m31:~/Documents/JCMMJ/v3-distributed/client$ cd ..
swarch@104m31:~/Documents/JCMMJ/v3-distributed$ cd ..
swarch@104m31:~/Documents/JCMMJ$ bash v3-distributed/start-node.sh client
Jun 07, 2026 8:42:45 PM co.icesi.project.v3.client.ClientApp main
INFO: Submitting task v3-flex-demo through Broker
Task: v3-flex-demo
Success: true
Message: Distributed calculation completed with 10 active workers and 10/10 completed partitions
Output: output/resultados-v3.csv
Records: 1356
ElapsedMs: 64899
swarch@104m31:~/Documents/JCMMJ$ ls
data v3-distributed
swarch@104m31:~/Documents/JCMMJ$ cd data/
swarch@104m31:~/Documents/JCMMJ/data$ ls
swarch@104m31:~/Documents/JCMMJ/data$ cd ..
swarch@104m31:~/Documents/JCMMJ$ cd v3-distributed/
swarch@104m31:~/Documents/JCMMJ/v3-distributed$ ls
client deploy.env start-client.sh start-node.sh start-visualization.sh visualization
swarch@104m31:~/Documents/JCMMJ/v3-distributed$ cd ..
swarch@104m31:~/Documents/JCMMJ$ ls out
ls: cannot access 'out': No such file or directory
swarch@104m31:~/Documents/JCMMJ$ ls output
ls: cannot access 'output': No such file or directory
swarch@104m31:~/Documents/JCMMJ$ cd ..
swarch@104m31:~/Documents/JCMMJ$ ls
Bugambilia.txt JCMMJ
swarch@104m31:~/Documents/JCMMJ$ cd ..
swarch@104m31:~/Documents/JCMMJ$ ls
Desktop Documents Downloads exam-node Music Pictures Public snap Templates Videos
swarch@104m31:~/Documents/JCMMJ$ cd Documents/
swarch@104m31:~/Documents/JCMMJ$ ls
Bugambilia.txt JCMMJ
swarch@104m31:~/Documents/JCMMJ$ cd JCMMJ/
swarch@104m31:~/Documents/JCMMJ/JCMMJ$ bash v3-distributed/start-node.sh client
Jun 07, 2026 8:48:36 PM co.icesi.project.v3.client.ClientApp main
INFO: Submitting task v3-flex-demo through Broker
Task: v3-flex-demo
Success: true
Message: Distributed calculation completed with 10 active workers and 10/10 completed partitions
Output: output/resultados-v3.csv
Records: 1356
ElapsedMs: 65429
swarch@104m31:~/Documents/JCMMJ$ ls
data v3-distributed
swarch@104m31:~/Documents/JCMMJ$ cd v3-distributed/
swarch@104m31:~/Documents/JCMMJ/v3-distributed$ ls
client deploy.env start-client.sh start-node.sh start-visualization.sh visualization
swarch@104m31:~/Documents/JCMMJ/v3-distributed$

```

Nota. La captura valida dos hechos: el worker logra registrarse con el master después de varios intentos, y el cliente recibe una respuesta exitosa de la tarea v3-flex-demo: Success: true, 10 workers activos, 10/10 particiones completadas, 1356 registros y salida output/resultados-v3.csv. La búsqueda posterior de carpetas out/output en el cliente refuerza que la salida consolidada debe verificarse en el nodo master.

Arquitectura del sistema

La solución se organiza como una arquitectura distribuida por componentes. Cada módulo tiene una responsabilidad específica y se comunica con los demás mediante interfaces remotas definidas en el módulo slice.

Tabla 5

Responsabilidades de los componentes de la Versión 3

Componente	Responsabilidad
Client	Construye la tarea de cálculo y la envía al broker.
Broker	Recibe la solicitud del cliente y la reenvía al master.
Master	Registra workers, asigna particiones, recibe resultados parciales y genera el CSV final.
Worker	Procesa una partición y calcula velocidades parciales.
Partitioner	Divide el dataset en particiones usando la clave busId lineId.
Visualization	Recibe eventos y muestra el comportamiento del sistema y las posiciones de buses.
Slice	Define los contratos remotos usados por los componentes.

Nota. Los componentes separan coordinación, procesamiento y observabilidad, lo cual facilita explicar y probar el sistema distribuido.

Los patrones arquitectónicos más relevantes son Broker, Master-Worker, particionamiento de datos, Map-Reduce simplificado y comunicación orientada a eventos. Estos patrones son reconocibles en la estructura del código, en los nombres de los módulos y en el flujo de ejecución: client -> broker -> master -> workers -> master -> resultado.

Despliegue

El despliegue se realizó sobre varios computadores del laboratorio. El equipo de visualización y cliente ejecuta la interfaz y lanza la tarea. El equipo coordinador ejecuta master, broker y particionador. Los demás equipos ejecutan workers. La configuración se centraliza en el archivo v3-distributed/deploy.env.

En la configuración actual se usan los puertos 11000 para broker, 11001 para master y 11003 para visualización. Los workers usan puertos derivados de `WORKER_BASE_PORT=11010`. Esta configuración evita conflictos con procesos anteriores y facilita repetir el despliegue.

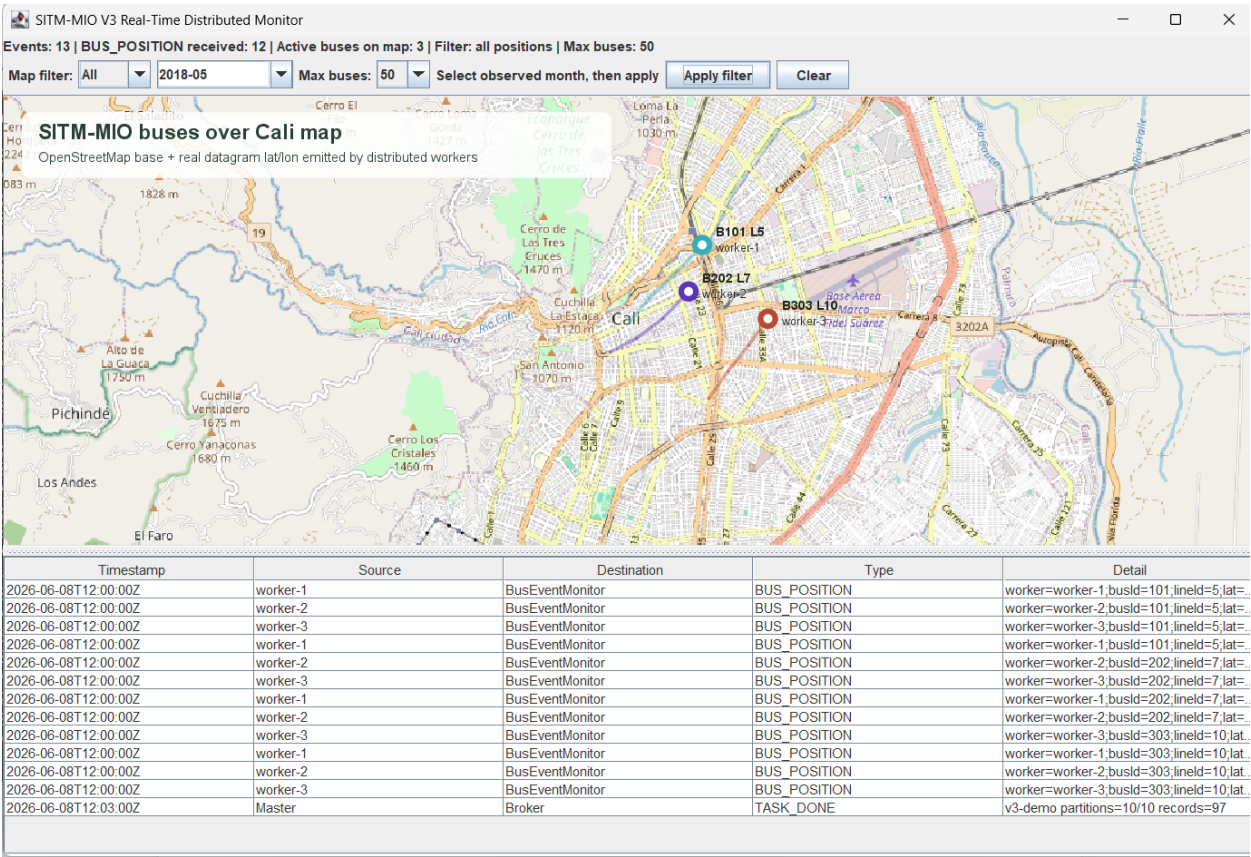
Visualización

La visualización se diseñó como una herramienta de apoyo para observar el comportamiento del sistema distribuido. Los workers publican eventos `BUS_POSITION` con el identificador del bus, la ruta, las coordenadas, la fecha del datagrama y el worker que procesó el evento.

Para evitar que el mapa se sature, la interfaz permite filtrar por mes y limitar la cantidad máxima de buses renderizados. Esta decisión no altera el cálculo final, porque el procesamiento completo sigue ejecutándose en los workers; únicamente se controla cuánta información se dibuja en pantalla durante la demostración. Cuando se usan mapas de OpenStreetMap, la atribución debe ser visible para los usuarios del mapa o del material producido (OpenStreetMap Foundation, 2022).

Figura 2

Ejecución distribuida de la Versión 3 que muestra la parte de la visualización de los buses, en el mapa de Cali, basado en el procesamiento realizado por los workers.



Validación

La implementación fue validada mediante compilación Gradle y ejecución del flujo distribuido completo. La validación incluyó el registro de workers, el envío de la tarea desde el cliente, la consolidación de resultados en el master y la recepción de eventos en la visualización.

Comandos de validación ejecutados:

```
.\gradlew.bat :v3-distributed:visualization:installDist :v3-distributed:client:installDist :v3-distributed:master:installDist :v3-
```

```
distributed:broker:installDist :v3-distributed:worker:installDist :v3-
distributed:partitioner:installDist
bash v3-distributed/start-node.sh visualization
bash v3-distributed/start-node.sh worker N
bash v3-distributed/start-node.sh coordination
bash v3-distributed/start-node.sh client
```

Limitaciones metodológicas

La evidencia disponible permite validar el flujo funcional y distribuido, pero no permite una evaluación completa de rendimiento. Para afirmar mejoras cuantitativas sólidas sería necesario ejecutar V1, V2 y V3 con el mismo dataset, la misma máquina o infraestructura controlada, varias repeticiones por versión y métricas separadas de I/O, cálculo, red y consolidación. Sin esa matriz experimental, la comparación debe presentarse como técnica y arquitectónica, no como benchmark definitivo.

También debe considerarse que el particionamiento por busId|lineId protege la correctitud del cálculo, pero puede producir particiones desbalanceadas si algunas rutas o buses concentran muchos más datagramas que otros. En ese caso, el tiempo total de la ejecución distribuida queda limitado por el worker más lento. De igual forma, se puede afirmar que en cuanto a la ejecución del 4Pilot la V3 es mejor, puesto que inicialmente la razón por la que se realiza una versión distribuida es por el cuello de botella que se genera al procesar el datagrama completo en las versiones monolíticas.

Conclusiones

La Versión 3 permite procesar el dataset piloto con una arquitectura más escalable que las versiones anteriores. La división del trabajo en particiones reduce la carga individual de cada equipo y permite experimentar con distintas cantidades de workers.

El particionamiento por busId|lineId fue una decisión clave para mantener la correctitud, porque conserva juntas las posiciones que deben compararse. Además, el procesamiento por streaming redujo el consumo de memoria y permitió que los workers procesaran particiones grandes de forma más estable.

La visualización complementa la solución porque permite observar la ejecución distribuida y los recorridos de buses sin intervenir en el resultado final. En conjunto, la Versión 3 responde mejor a los drivers de performance y escalabilidad planteados para el proyecto, aunque su desempeño debe medirse de forma controlada antes de presentar conclusiones cuantitativas fuertes frente a V1 y V2.

Referencias

OpenStreetMap Foundation. (2022). Licence/Attribution guidelines. OpenStreetMap Foundation.

https://osmfoundation.org/wiki/Licence/Attribution_Guidelines

Sinnott, R. W. (1984). Virtues of the Haversine. Sky and Telescope, 68(2), 159.

ZeroC, Inc. (2018). Ice 3.7.1 documentation: Distributed programming with Ice.

<https://download.zeroc.com/ice/3.7/Ice-3.7.1.pdf>